# Atividade prática extra dos Encontros 1 e 2

*Atividade avaliativa com front-end fornecido*

## Mural de Avisos Acadêmicos

**Valor sugerido: 1,0 ponto**

**Organização: grupos**

**Entrega: código e evidências de execução**

## Objetivo

Construir um back-end FastAPI para o front-end fornecido. A atividade verifica arquitetura Full Stack, ambiente virtual, rotas HTTP, JSON, Pydantic, Fetch API, CORS, Swagger e diagnóstico pelo Network.

## Recurso da aplicação

O sistema trabalhará com o recurso `Aviso`, diferente do Sistema de Reservas desenvolvido nas aulas.

Cada aviso deverá possuir:

- `id`, gerado pela API;
- `titulo`;
- `mensagem`;
- `autor`;
- `prioridade`, com os valores `normal` ou `importante`;
- `publicado_em`, gerado pela API.

Os dados poderão permanecer em uma lista Python. SQLite não é necessário nesta atividade.

## Material fornecido

O pacote contém `index.html`, `styles.css` e `app.js` prontos. Execute o front-end na porta 5500. O JavaScript já espera uma API em `http://127.0.0.1:8000/api/avisos`.

## Requisitos obrigatórios

1. Criar a pasta `backend` com `main.py`, `models.py` e `requirements.txt`.
2. Criar e utilizar um ambiente virtual.
3. Implementar um modelo Pydantic de entrada e outro de saída.
4. Validar tamanhos mínimos e máximos compatíveis com o formulário.
5. Restringir `prioridade` a `normal` ou `importante`.
6. Configurar CORS para o front-end executado na porta 5500.
7. Implementar as duas rotas abaixo.
8. Utilizar códigos HTTP adequados.
9. Demonstrar um caso de sucesso e um caso de falha.

## Contrato mínimo da API

| Operação | Método e rota | Resposta esperada |
|---|---|---|
| listar avisos | GET /api/avisos | 200 e lista JSON |
| publicar aviso | POST /api/avisos | 201 e aviso criado |

Exemplo de JSON enviado pelo front-end:

```
{
  "titulo": "Monitoria de programação",
  "mensagem": "A monitoria ocorrerá no laboratório 3 às 18 horas.",
  "autor": "Coordenação",
  "prioridade": "importante"
}
```

A resposta do POST deverá acrescentar `id` e `publicado_em`.

## Testes mínimos

| Caso | Resultado esperado |
|---|---|
| GET antes de publicar | 200 e lista vazia |
| POST com dados válidos | 201 e objeto com id |
| GET após publicar | 200 e aviso na lista |
| prioridade diferente das opções | 422 |
| mensagem muito curta ou ausente | 422 |

## Evidências para entrega

- interface exibindo ao menos um aviso;
- Swagger mostrando as duas rotas;
- Network mostrando um GET 200 e um POST 201;
- resposta 422 provocada pelo Swagger;
- arquivos do back-end do grupo.

## Sugestões para a solução

- use `Literal["normal", "importante"]` para a prioridade;
- use `Field` para os limites de texto;
- mantenha uma lista `avisos` em `main.py`;
- gere o identificador com base na quantidade de itens da lista;
- gere `publicado_em` com `datetime.now()`;
- no POST, converta o modelo de entrada com `model_dump()`;
- compare os nomes enviados em `app.js` com os campos do modelo Pydantic;
- use o Swagger antes de testar a integração com o navegador.

Não é necessário implementar banco de dados, edição, exclusão ou autenticação.